🛡️ Module 11: Reverse Engineering, Fuzzing and Binary Exploitation

```
# Exercise 06: Exploiting Libc to Bypass No Execute Stack and Obtaining Root Privileges
```

Scenario

Return-to-Libc (Ret2Libc) Exploitation is an advanced memory corruption technique used to bypass modern Non-Executable Stack (**NX / DEP**) protections. When a program stack is marked non-executable (`-z noexecstack`), injecting shellcode directly onto the stack will result in immediate execution termination via segmentation fault (`SIGSEGV`).

To bypass NX/DEP without shellcode execution on the stack:
1. **Identify Shared Library Base Addresses**: In Linux environments with ASLR disabled (**sysctl kernel.randomize_va_space=0**), the C standard library (`libc.so`) is mapped at static memory addresses across executions.
2. **Locate Target Functions**: Using GDB, determine the exact memory addresses of C library functions `system()` and `exit()` .
3. **Locate String Arguments**: Export the target shell command string (`/bin/sh`) into an environment variable (**export MYSHELL=/bin/sh**) and compute its precise memory pointer using GDB or custom environment pointer utilities (**printenv.c** / **getenv.c**).
4. **Construct Return-to-Libc Payload**: Overwrite the target function's return address (`EIP`) with the address of `system()` , followed by the return address of `exit()` (for clean termination), and pass the 32-bit pointer address to `/bin/sh` as the first argument.

In this exercise, we log into **Software-Test-Linux-32bit** (**student** / **studentpassword**), verify ASLR is disabled, compile a non-executable Set-UID binary (**retlib.c**), obtain `system()` and `exit()` function pointers in GDB, locate the `/bin/sh` environment variable pointer address, calculate stack frame offsets (**0x14** / 20 bytes between `&buffer` and `$ebp`), build the exploit payload (**exploit.c**), and execute `retlib` to obtain an elevated root shell (`#`).

Objective

The objective of this lab is to help students learn how to:
- Authenticate to 32-bit Linux target VM (**student** / **studentpassword**)
- Verify ASLR is disabled via **sysctl kernel.randomize_va_space** (value: `0`)
- Inspect vulnerable application **retlib.c** reading 300 bytes into a 12-byte stack buffer
- Compile binary with non-executable stack protection enabled: **gcc -fno-stack-protector -z noexecstack -o retlib retlib.c**
- Grant root SUID ownership on binary (**sudo chown root retlib** & **sudo chmod 4755 retlib**)
- Identify `system()` and `exit()` addresses in GDB via **p system** and **p exit**
- Export environment variable **export MYSHELL=/bin/sh** and locate memory pointer using **printenv.c**
- Inspect 32-bit function calling convention stack layouts for Ret2Libc payload alignment
- Calculate exact difference between `$ebp` and `&buffer` in GDB: **0x14** (20 bytes)
- Align payload offsets: `system()` at 24 bytes, `exit()` at 28 bytes, `/bin/sh` pointer at 32 bytes
- Execute crafted payload (`badfile`) against `retlib` to spawn an elevated root shell (`#`)

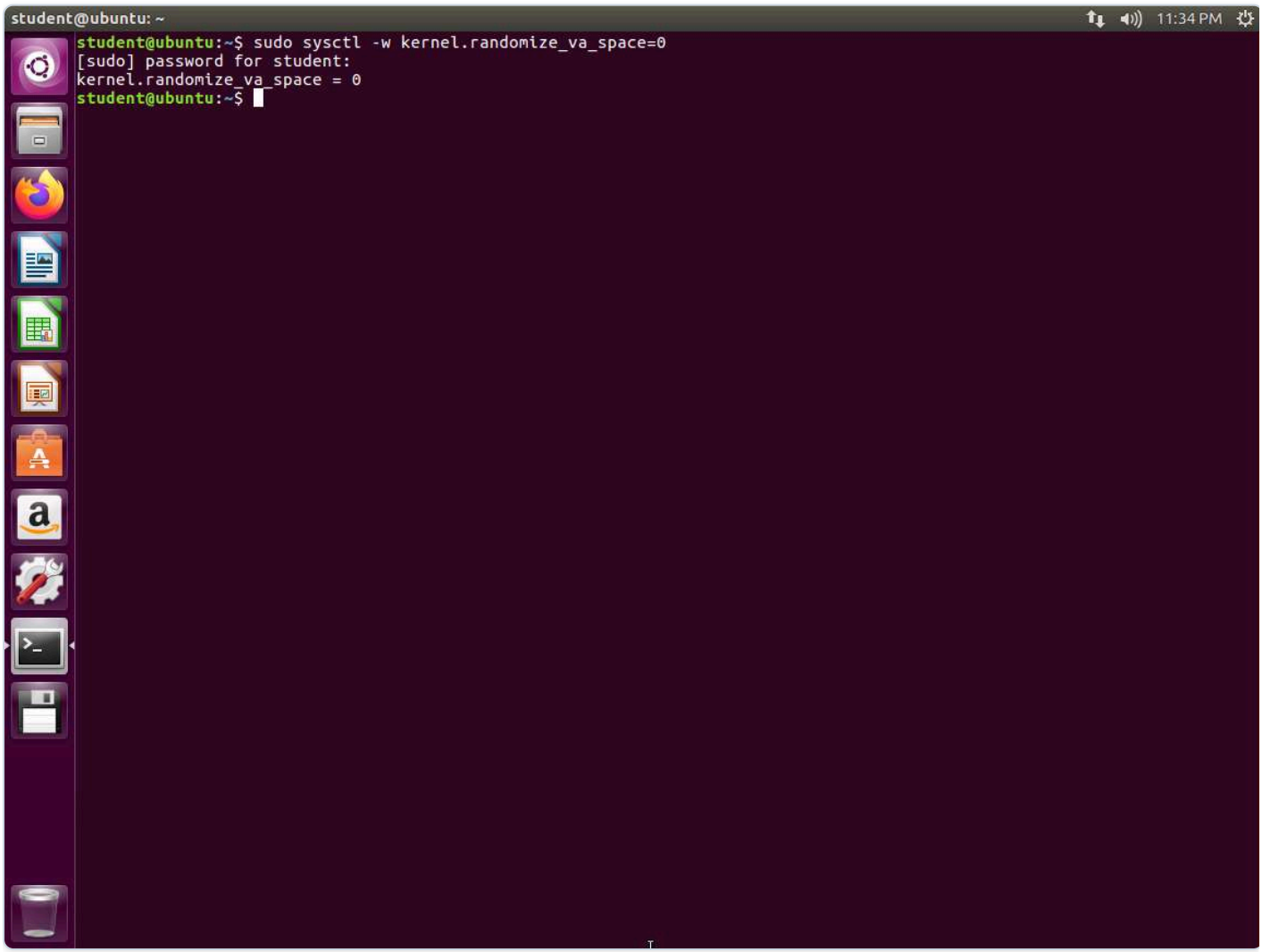
Task 1: Inspect Ret2Libc Targets, Calculate Stack Frame Differences, and Trigger Root Exploitation

In this task, we verify ASLR settings, compile `retlib` with non-executable stack (`-z noexecstack`), configure SUID permissions, inspect GDB function pointers, locate environment variable pointers, calculate stack offset differences (`0x14`), and spawn a root shell.

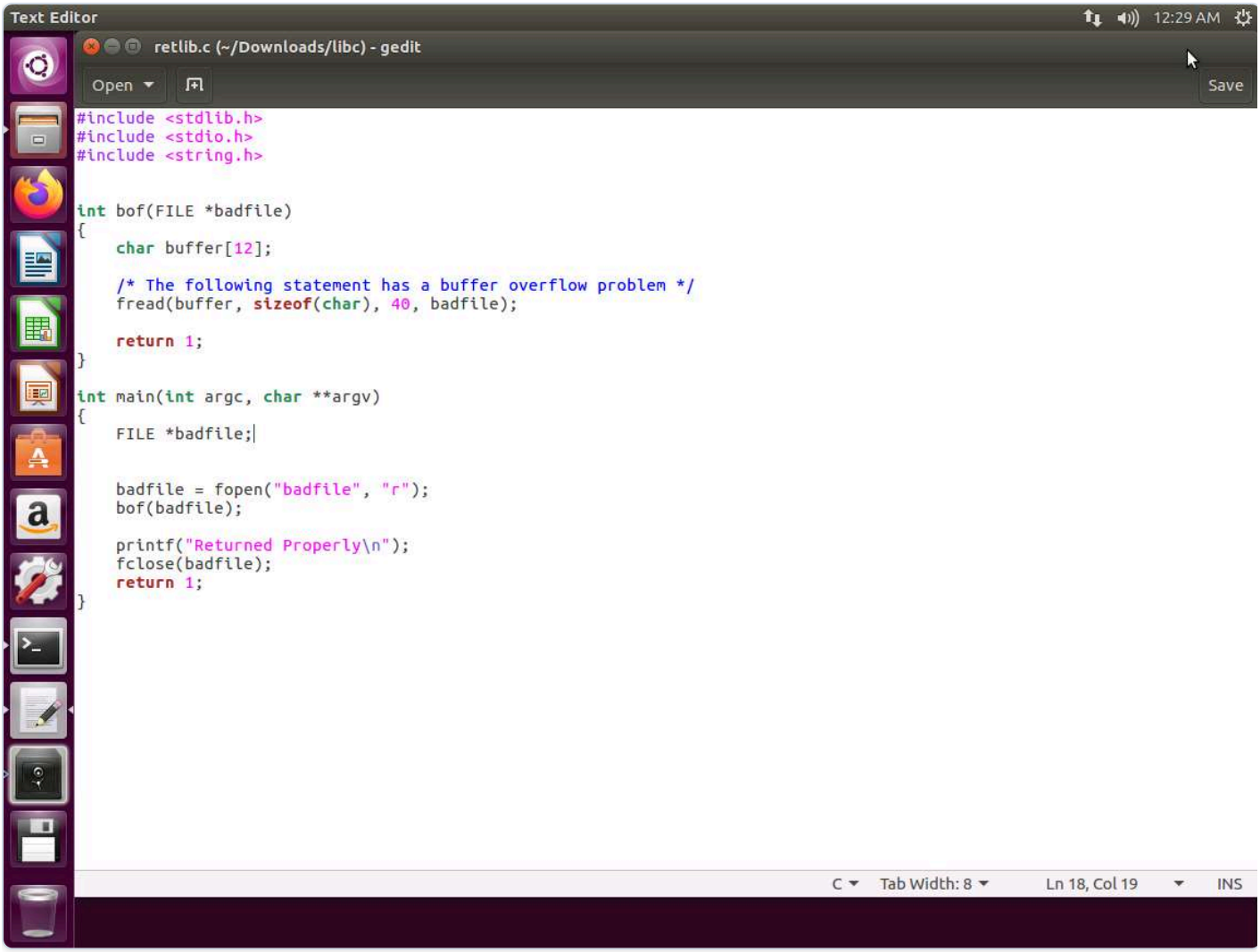
Lab Tasks

1. Log into **Software-Test-Linux-32bit** VM using credentials (**student** / **studentpassword**).

2. Verify that Address Space Layout Randomization (ASLR) is disabled:
   sysctl kernel.randomize_va_space
   (If enabled, set to 0 via `sudo sysctl kernel.randomize_va_space=0`).



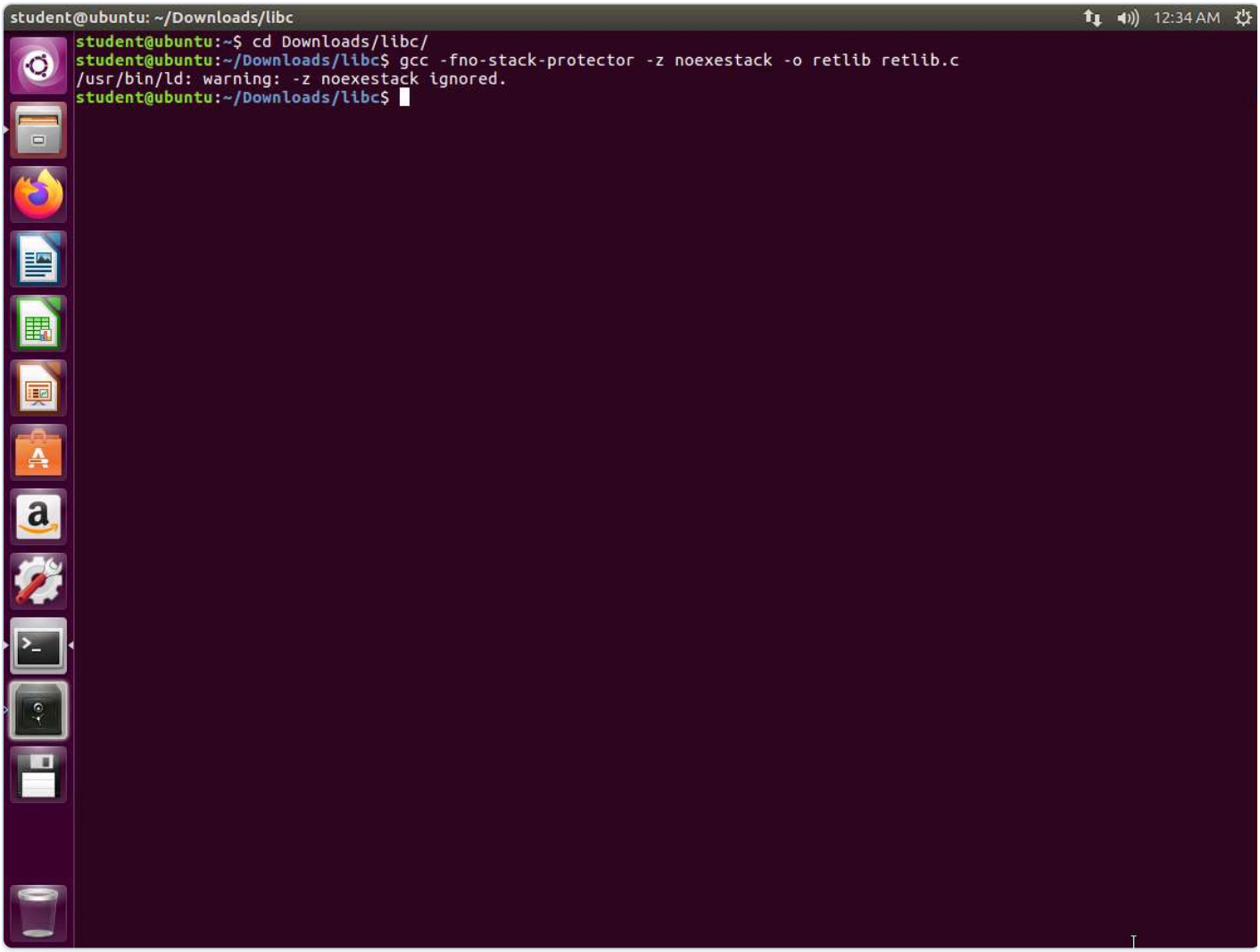
1. Inspect vulnerable Ret2Libc application source code **retlib.c**:
   Observe 300-byte read from `badfile` into a 12-byte stack buffer.



1. Navigate to directory and compile binary enforcing non-executable stack protection (`-z noexecstack`):

   cd Downloads/libc

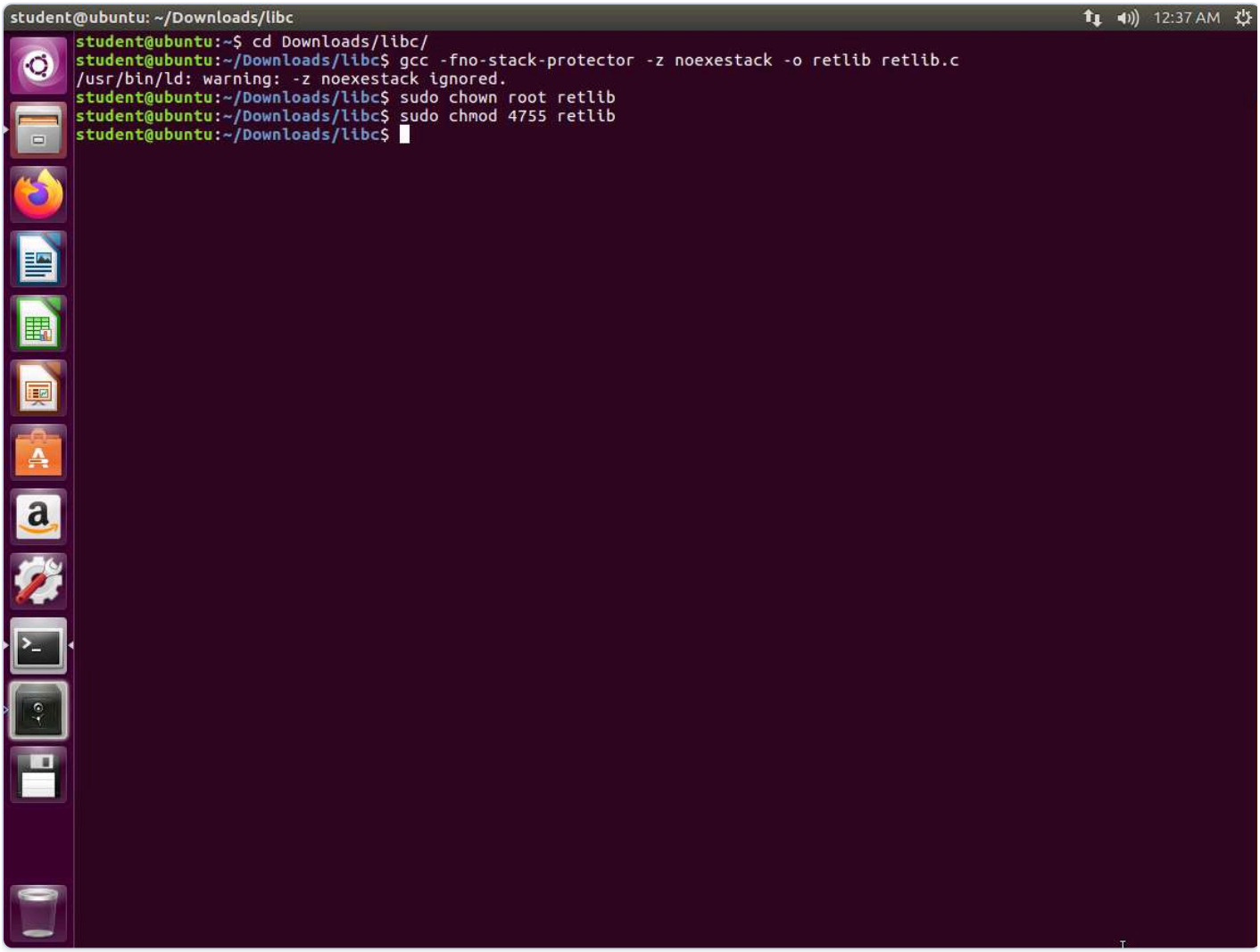
   gcc -fno-stack-protector -z noexecstack -o retlib retlib.c



1. Configure root ownership and set SUID bit on binary:

   sudo chown root retlib

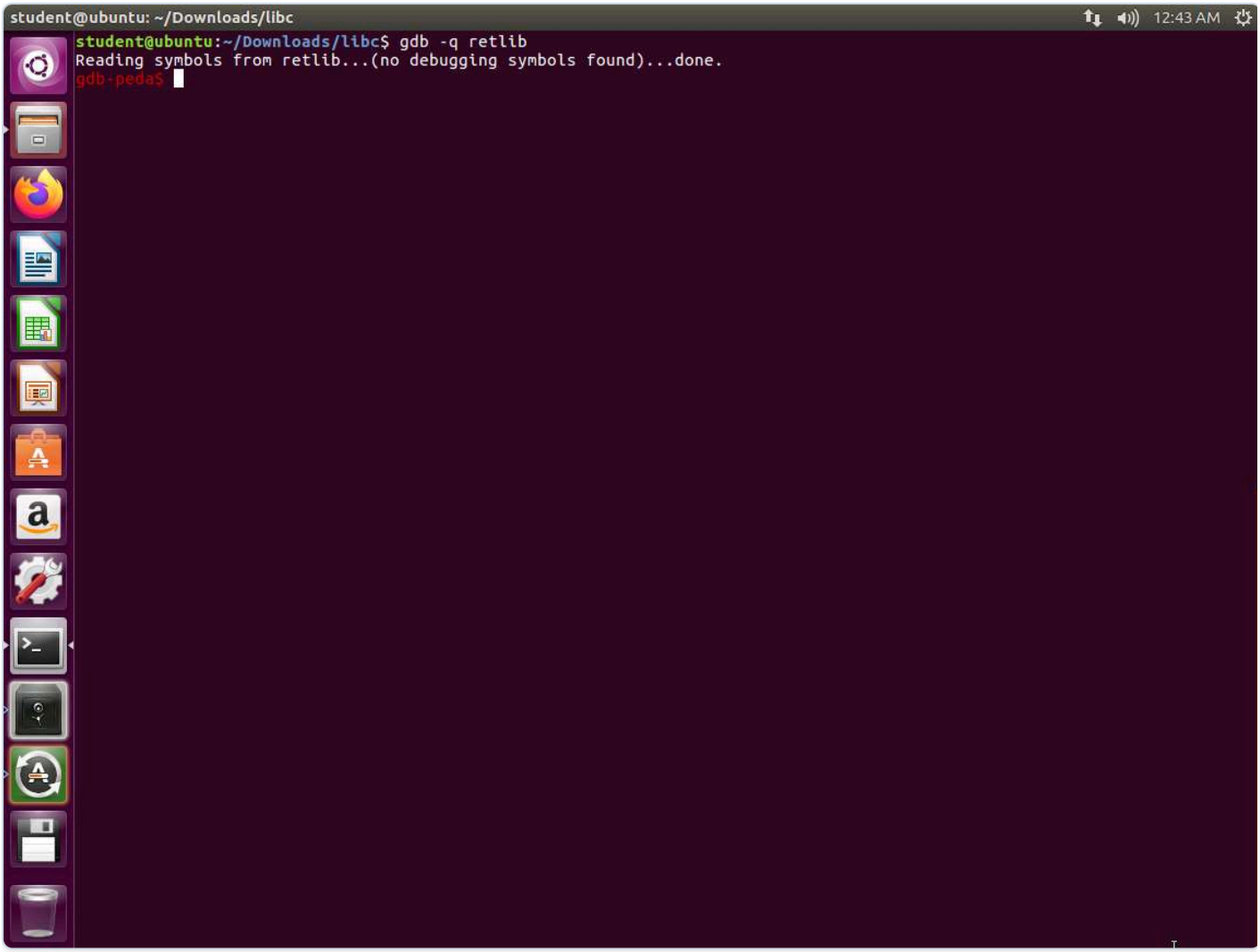
   sudo chmod 4755 retlib



1. Touch input file and launch GDB in quiet mode:

   touch badfile

   gdb -q retlib



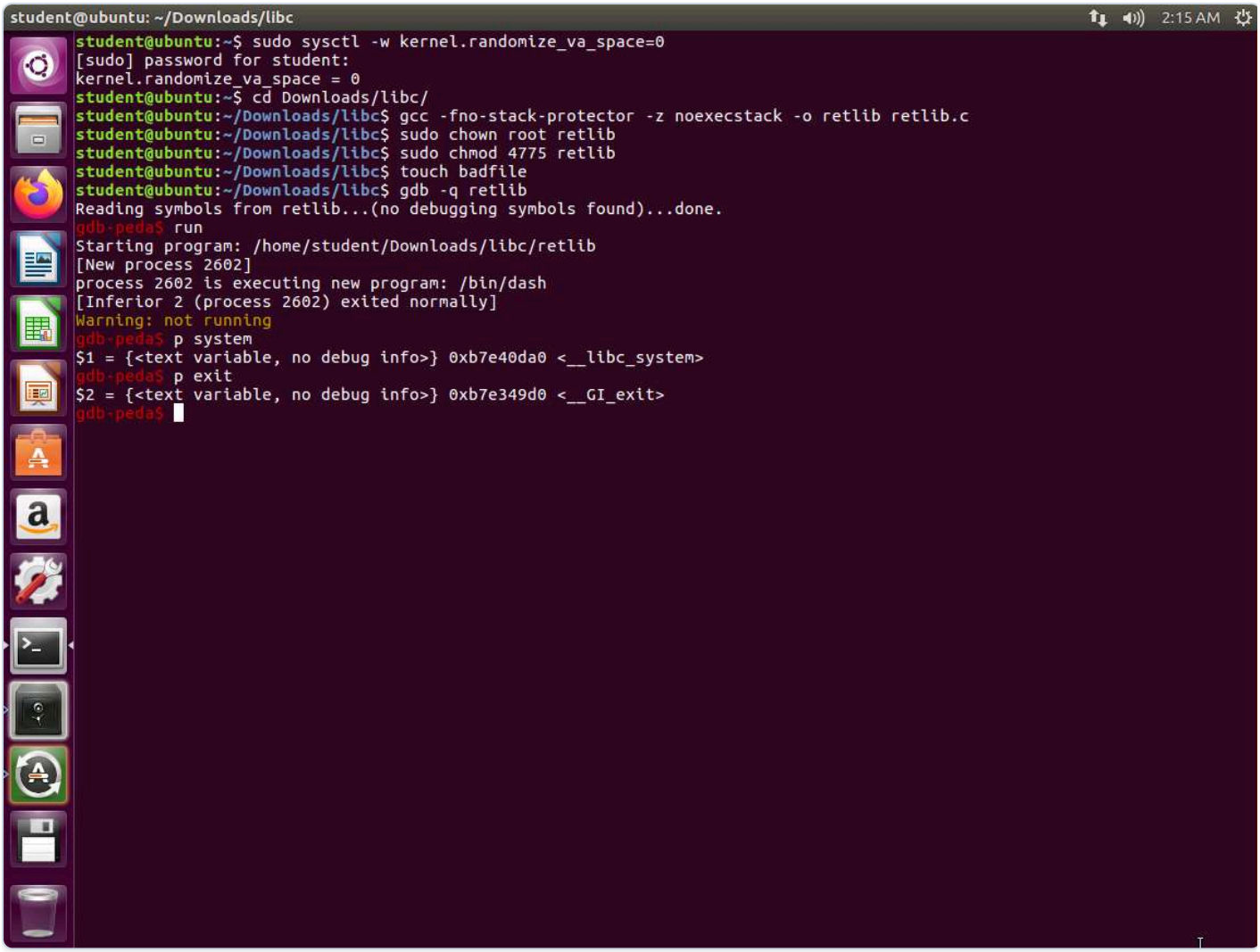
1. Run binary execution in GDB, print pointers for `system` and `exit`, and quit debugger:

   run

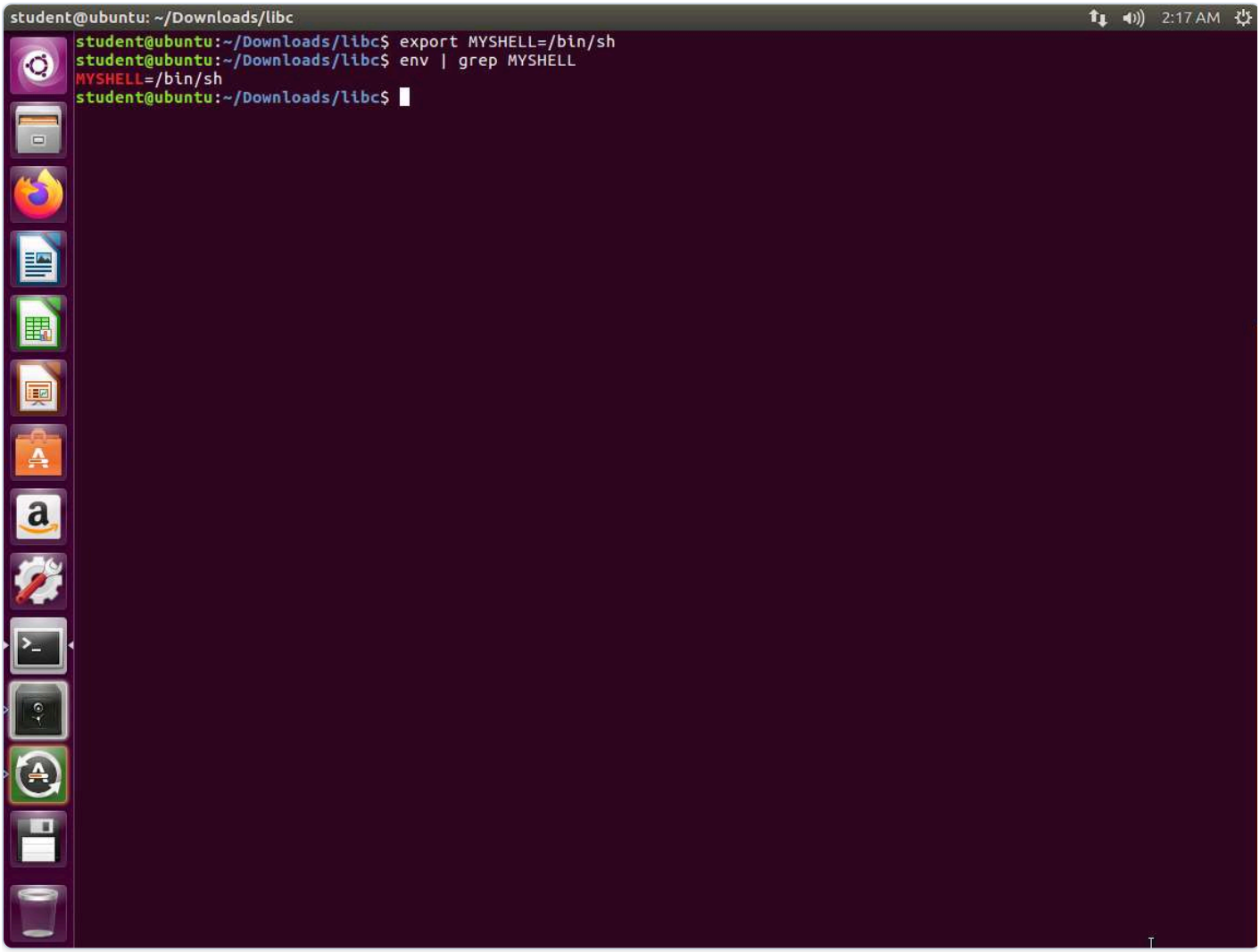
   p system

   p exit

   quit



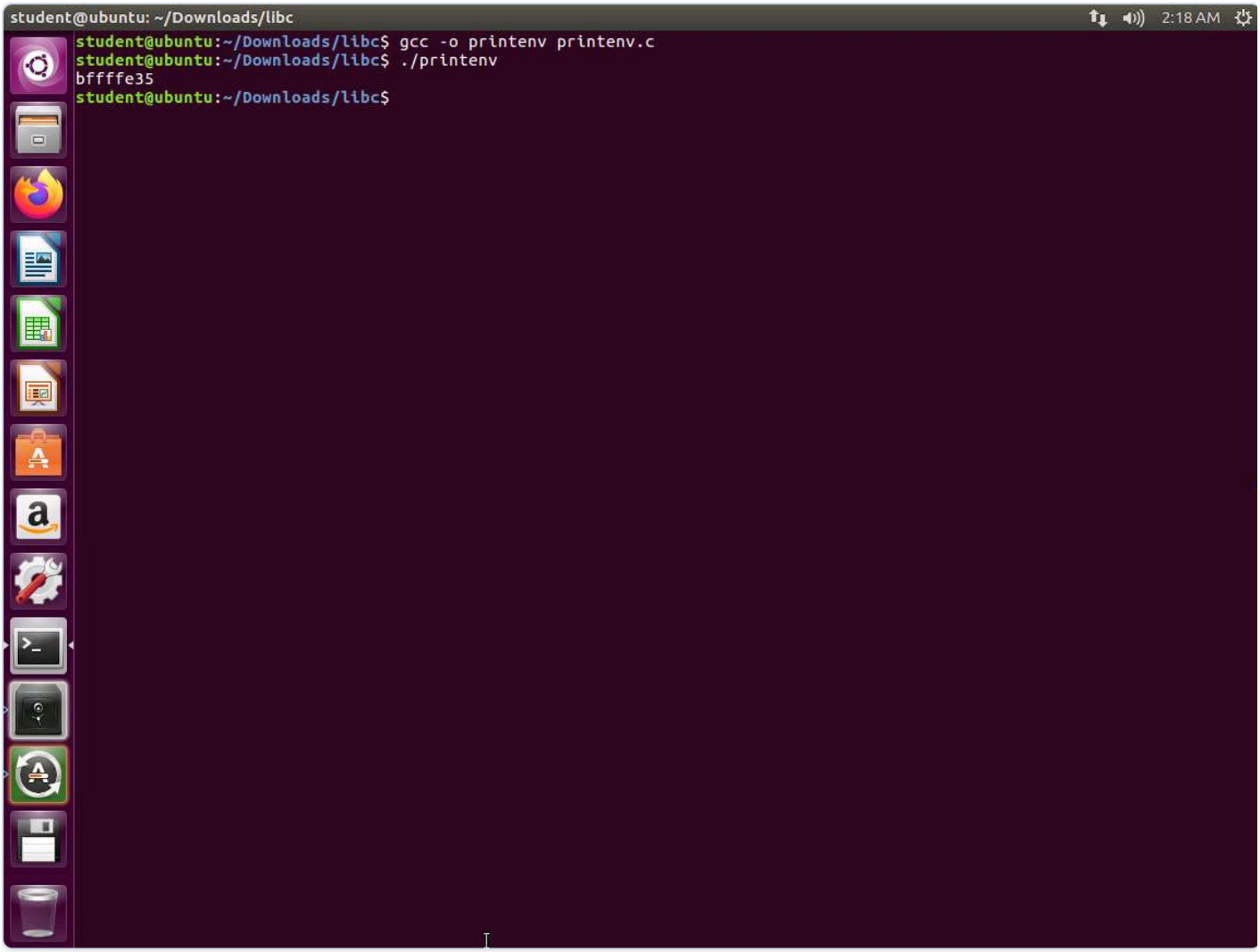
1. Export custom environment variable holding `/bin/sh` target command string:
   export MYSHELL=/bin/sh
   env | grep MYSHELL



1. Compile and execute environment variable pointer inspection utility **printenv.c**:

   gcc -o printenv printenv.c

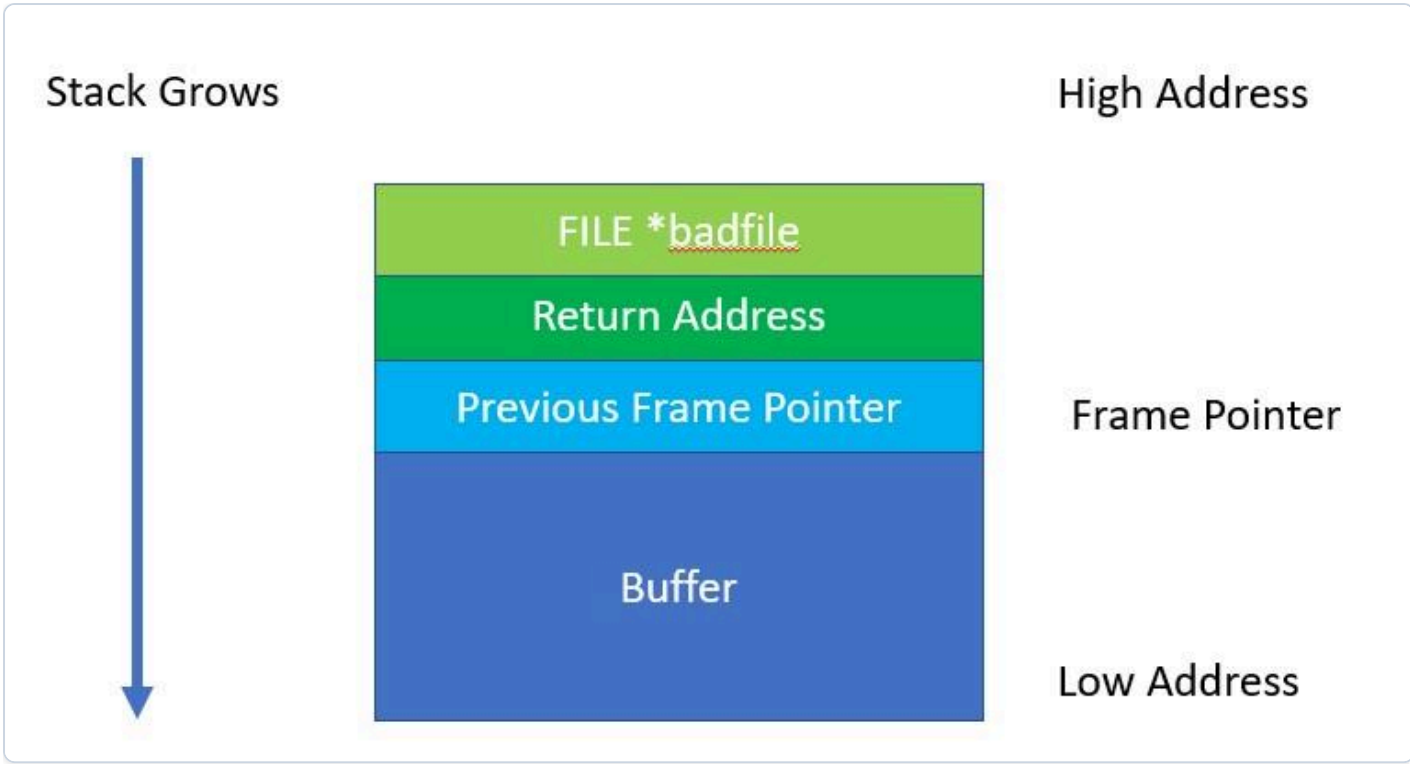
   ./printenv



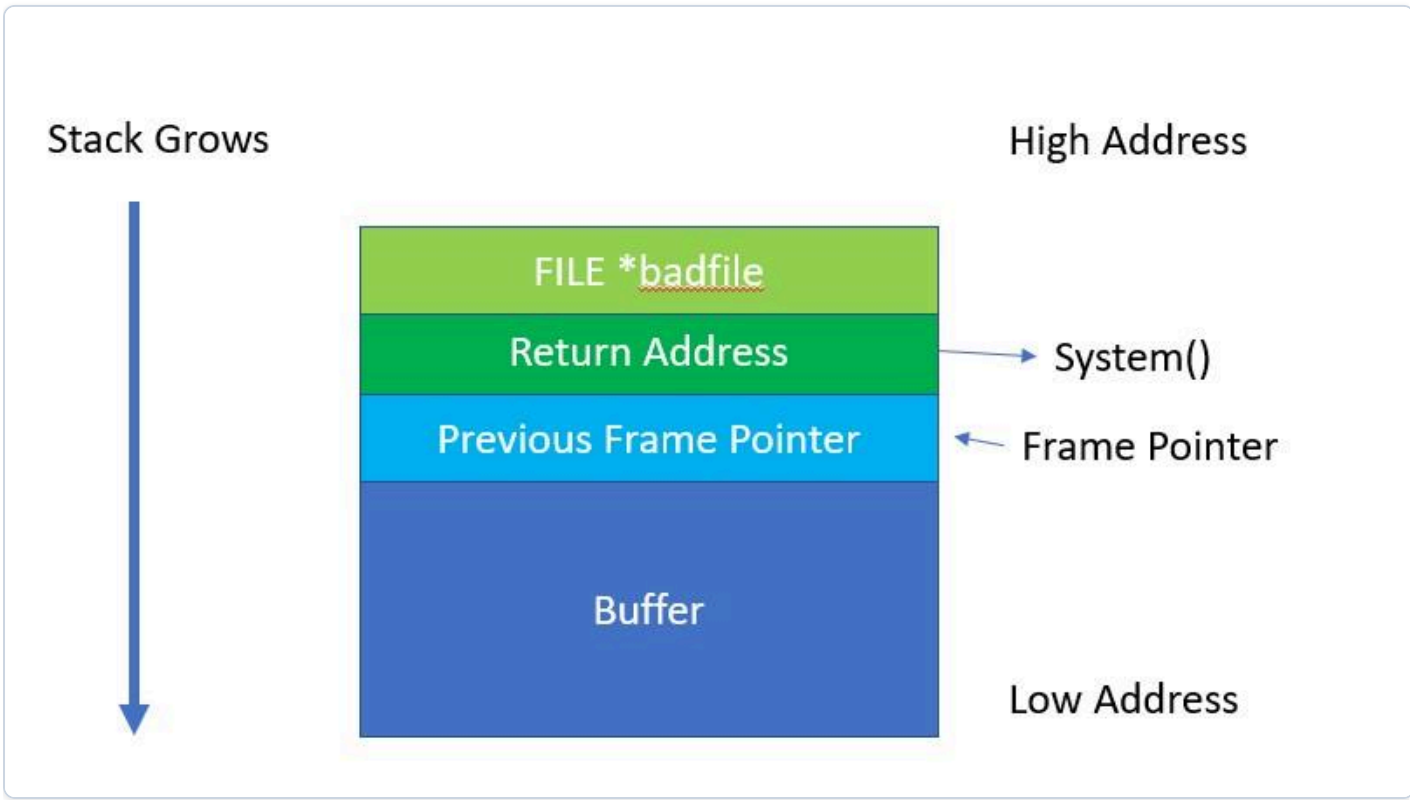
1. Review buffer overflow vulnerability in `bof()` method.

```
/* The following statement has a buffer overflow problem */
fread(buffer, sizeof(char), 300, badfile);
```

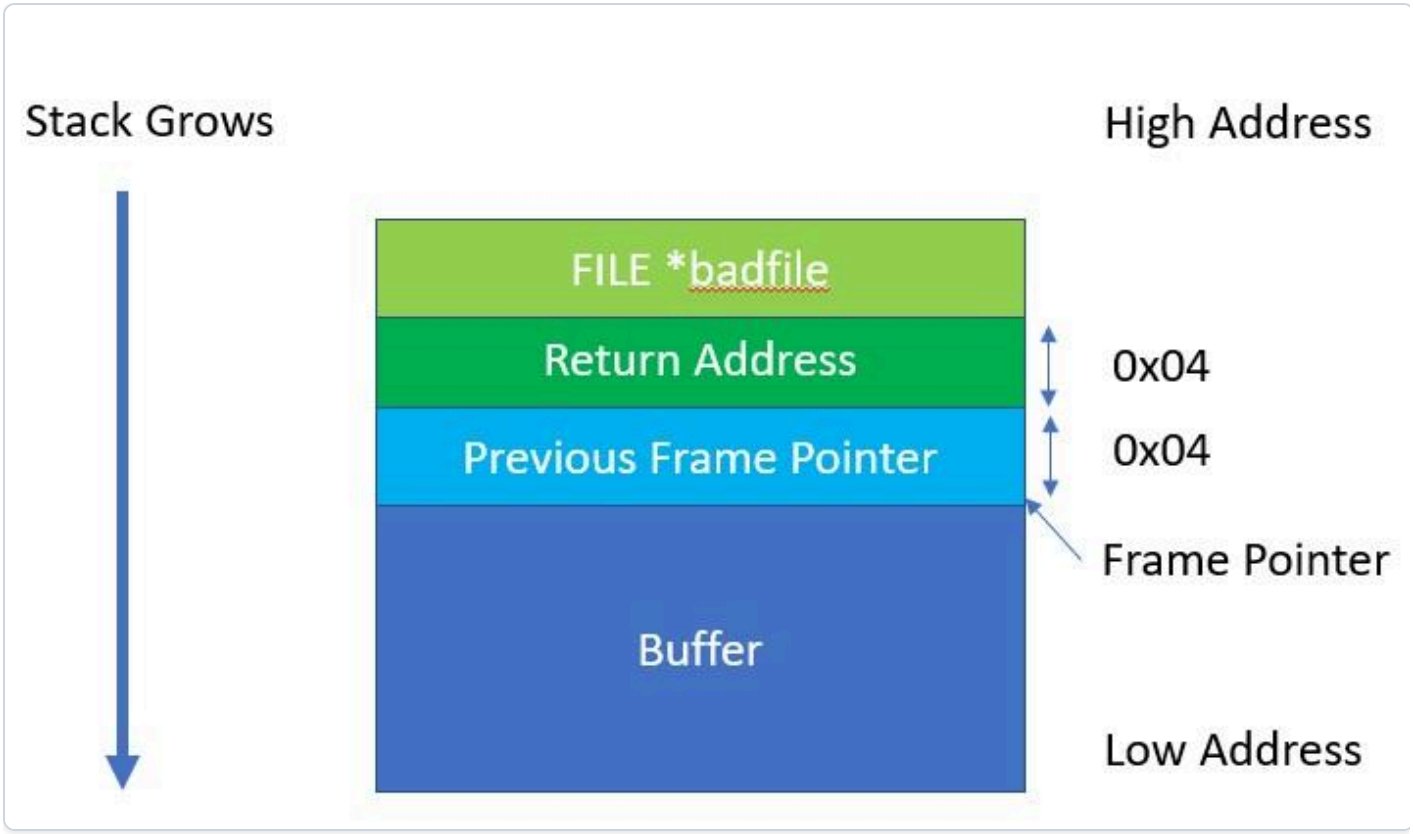
1. Review initial 32-bit stack frame architecture upon entering `bof()`.



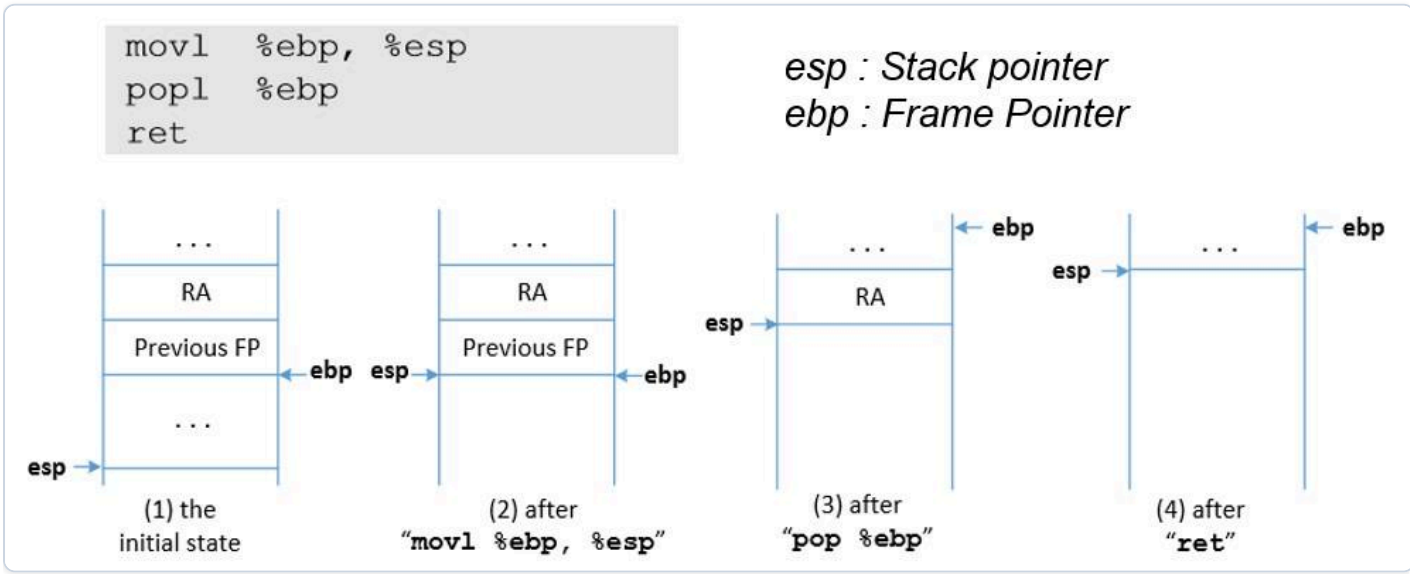
1. Review Return-to-Libc execution flow hijacking concept.



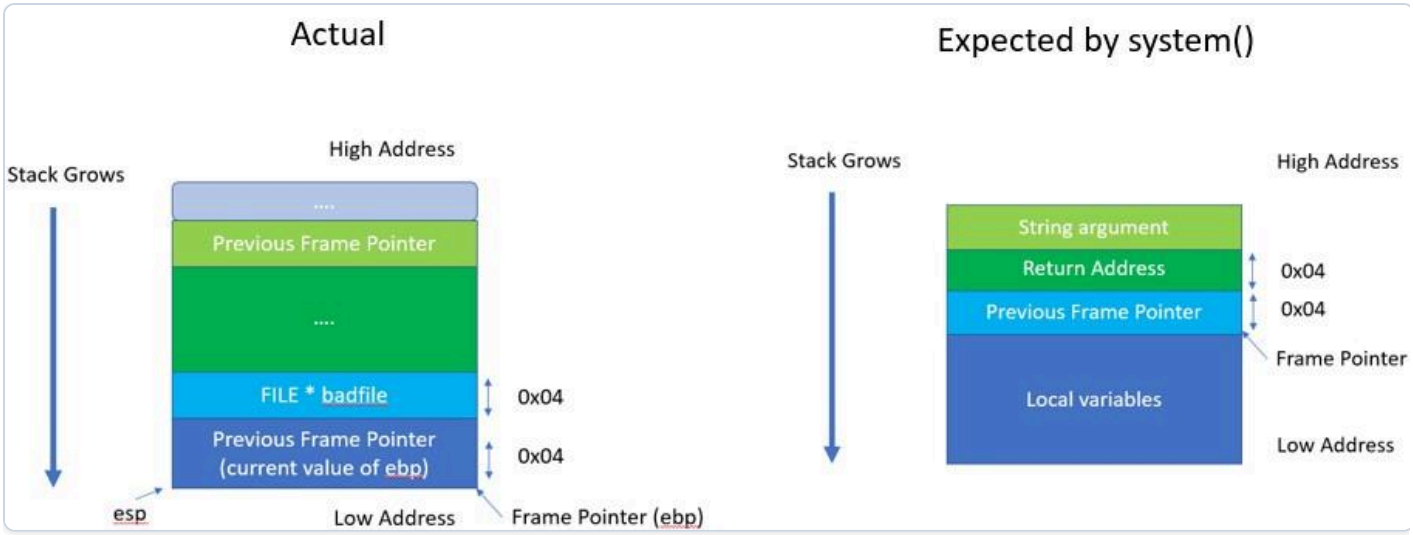
1. Review stack frame argument alignment requirements for system call parameter passing.



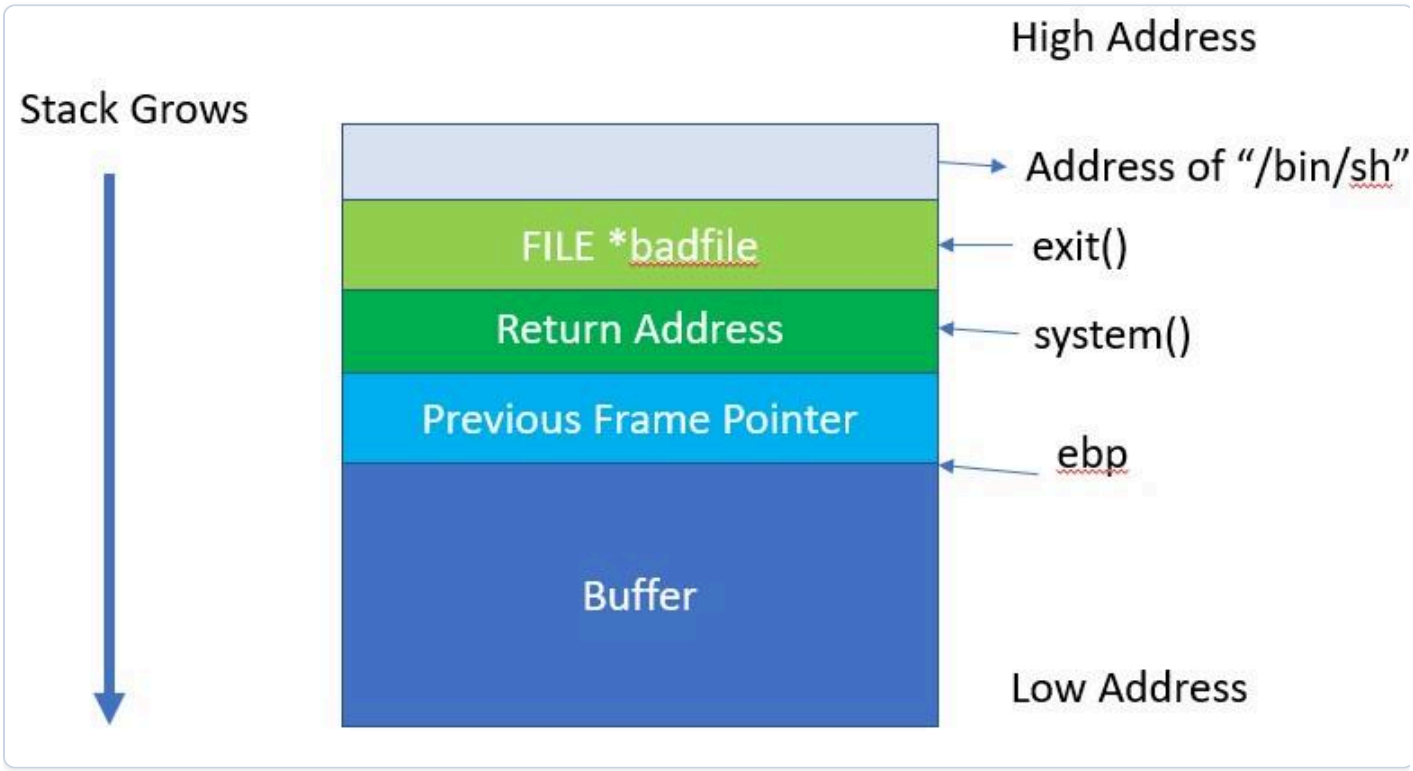
1. Review standard function epilogue disassembly instructions (`mov %ebp, %esp` / `pop %ebp`).



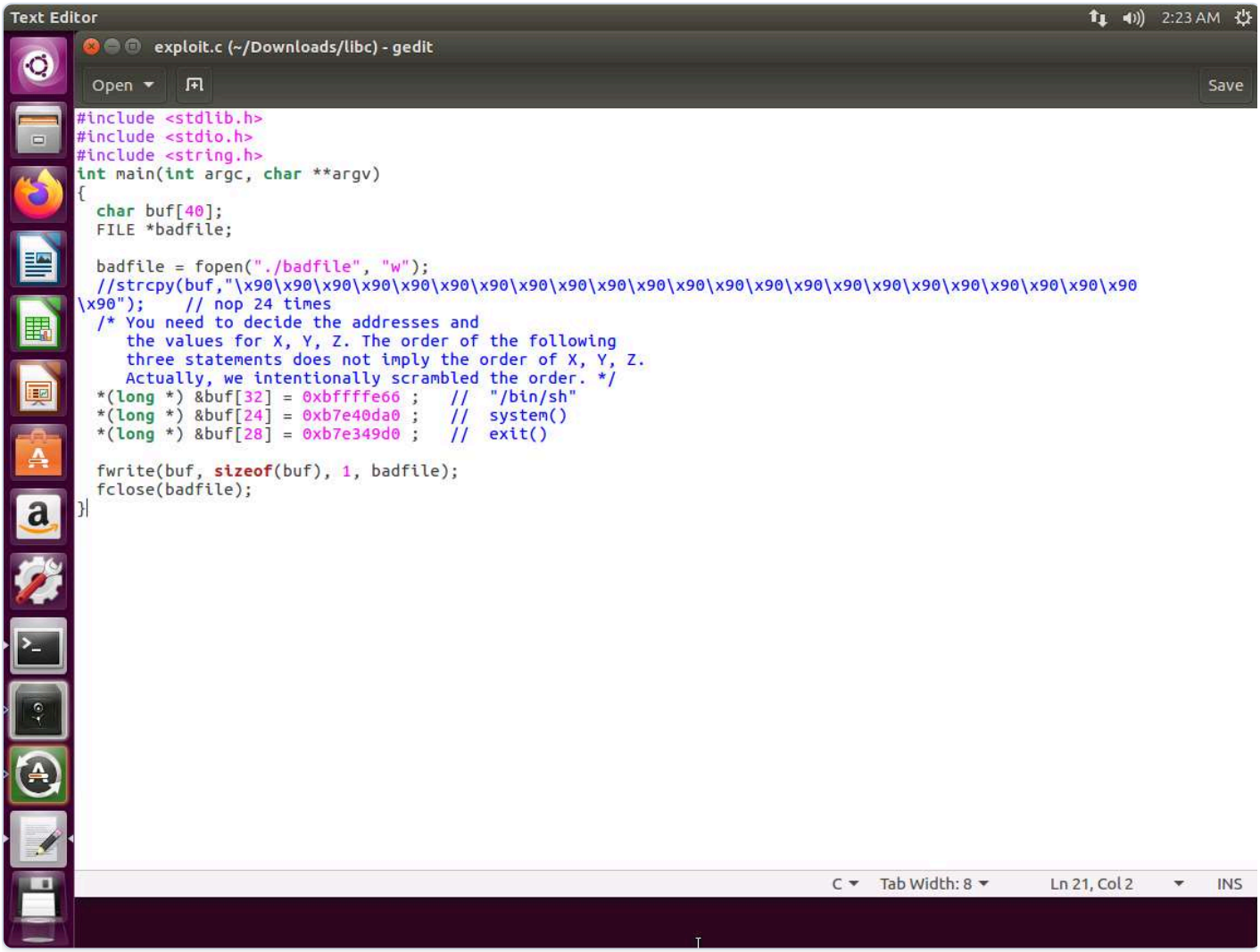
1. Review target stack frame structure when entering `system()` from hijacked return address.

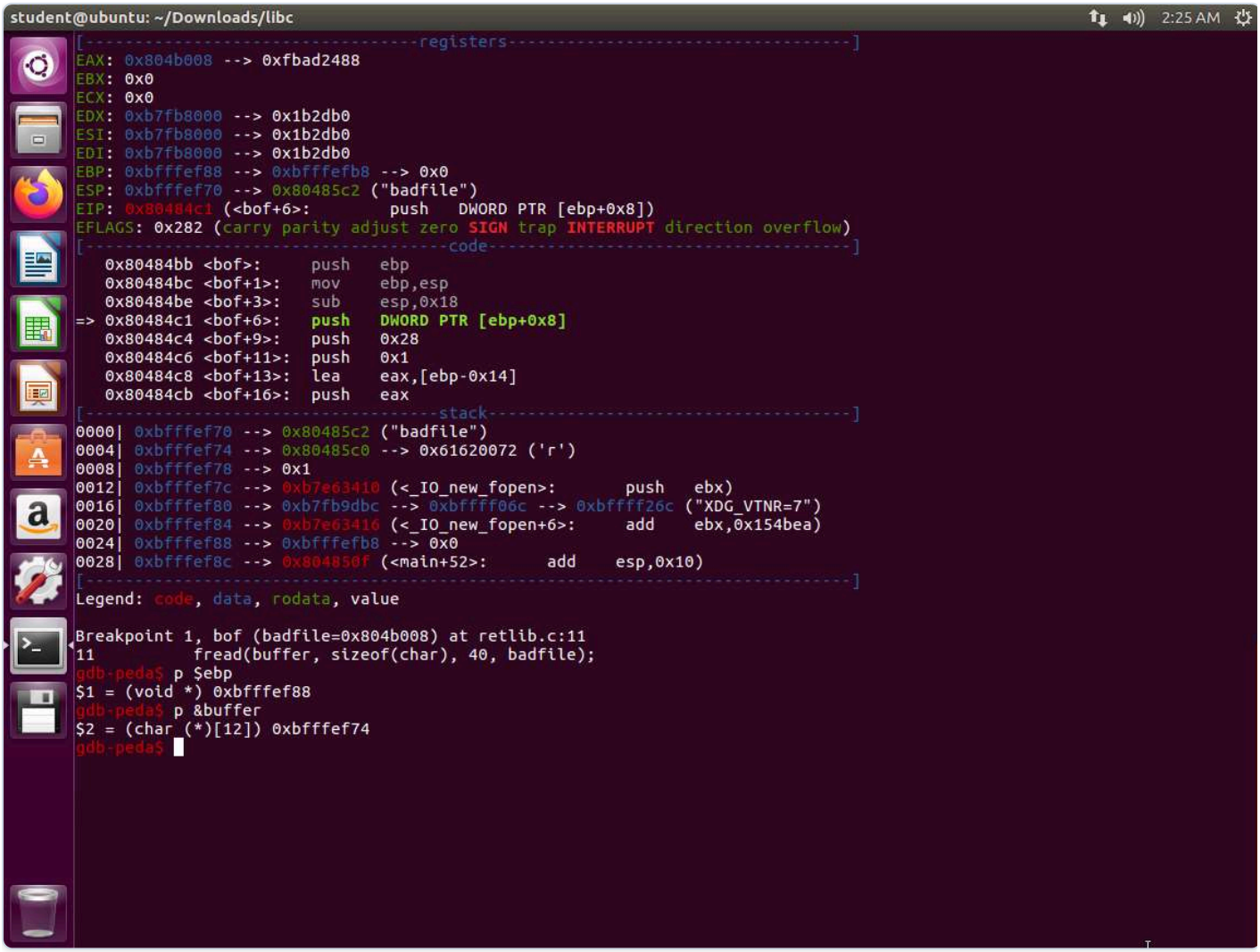


1. Review finalized Ret2Libc memory payload layout embedding `system()`, `exit()`, and `/bin/sh` pointer address.



1. Inspect exploit payload generator source file **exploit.c**.


Text Editor
2:23 AM
exploit.c (~/Downloads/libc) - gedit
Open
Save
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
int main(int argc, char **argv)
{
char buf[40];
FILE *badfile;
badfile = fopen("./badfile", "w");
//strcpy(buf,"\x90\x90\x90



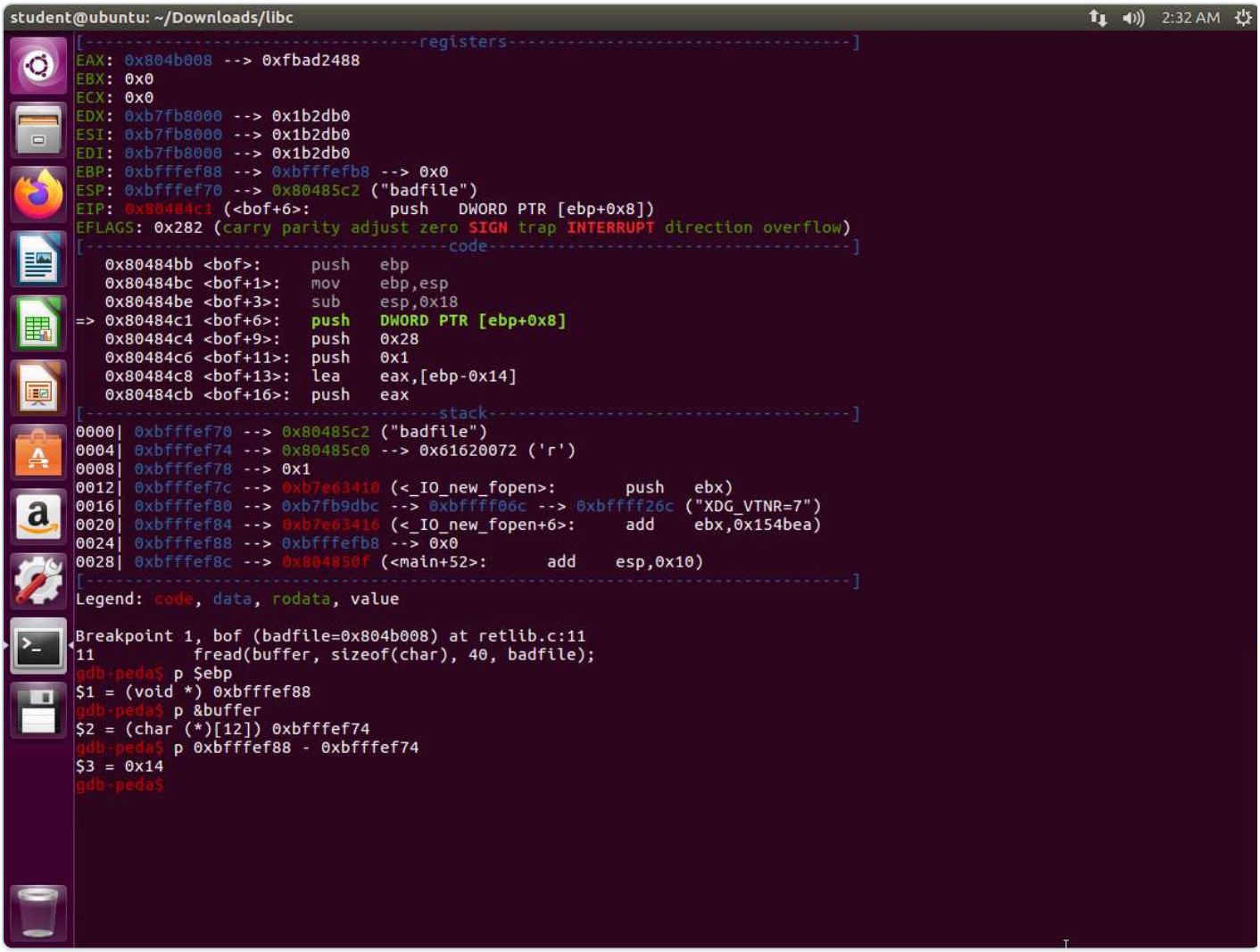
1. Verify memory address offset subtraction output in GDB:

   p $ebp - &buffer

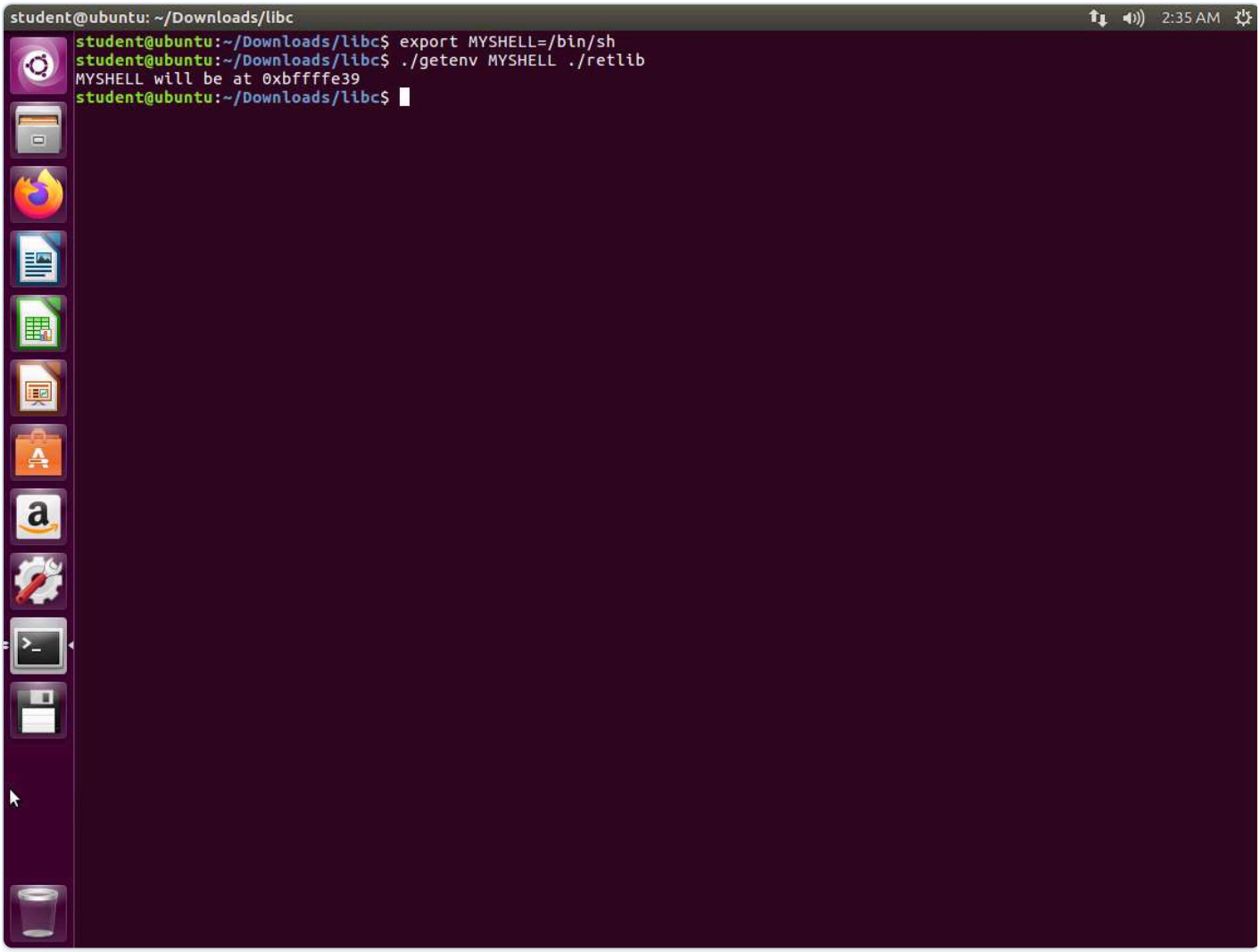
   Observe exact memory distance: **0x14** *(20 bytes).*

   Calculate target payload offsets:

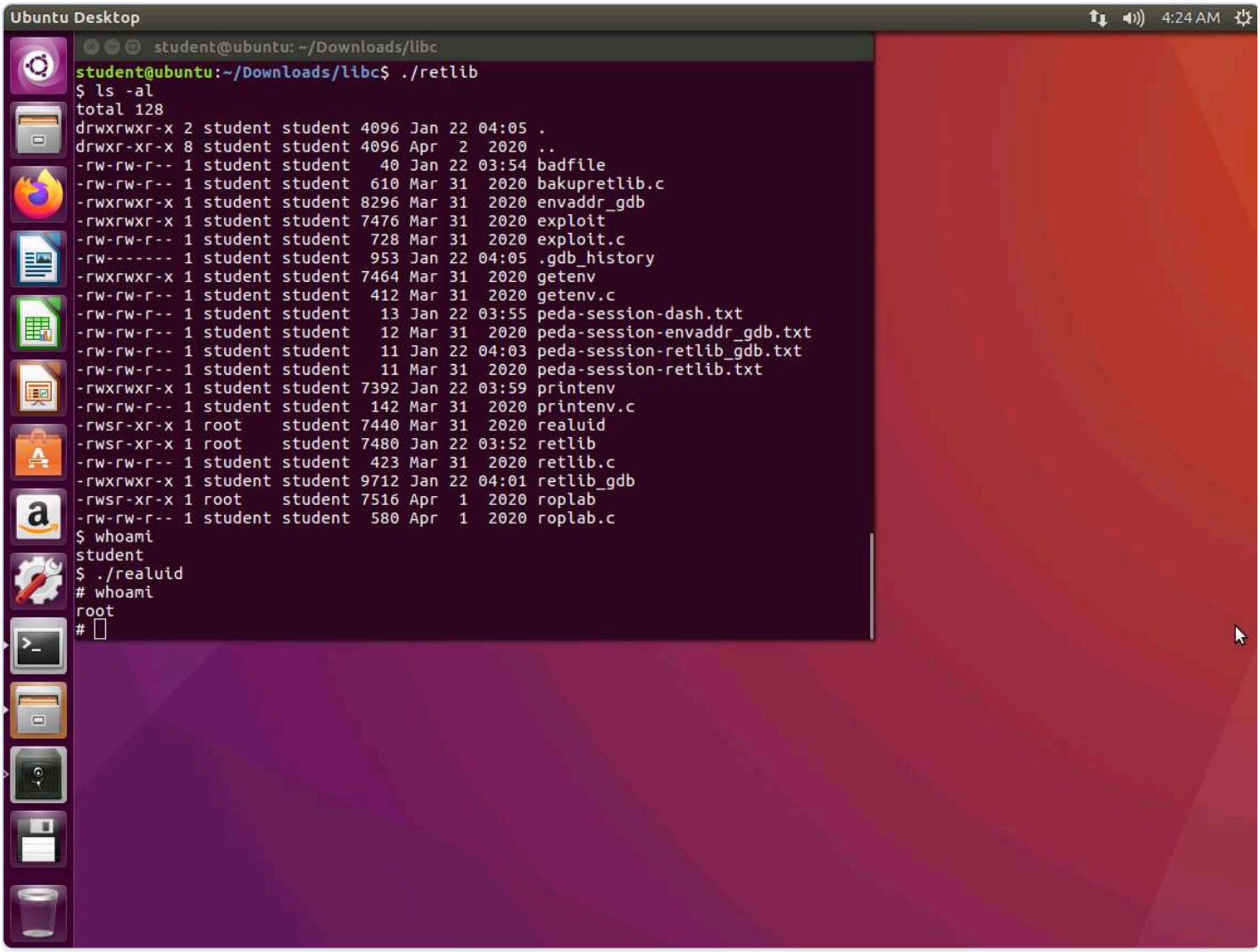
   - Buffer to EBP distance: **0x14** (20 bytes)
   - `system()` pointer offset: 20 + 4 = **24 bytes**
   - `exit()` pointer offset: 24 + 4 = **28 bytes**
   - `/bin/sh` string pointer offset: 28 + 4 = **32 bytes**



1. Audit environment variable location validation script **getenv.c**.



1. Execute compiled exploit payload generator, generate `badfile`, and run `retlib` to spawn an elevated root shell (`#`).



This concludes the lab exercise.

IMPORTANT: ❓ **Question 11.6.1.1** In Software-Test-Linux-32bit exploit a buffer overflow vulnerability in a program to bypass a non-executable stack by leveraging the libc library. Identify the difference of p $ebp and p &buffer addresses. [!SUCCESS] ✅ **Answer:** **0x14**